

D4-V4 — Search & Rescue: Heterogeneous Sensing

AI4RO2 (2026) — PDDL / PDDL+ project report

Author: Amr Magdy Mohamed Elsayed Abdalla • ID: S8082888

Repository: https://github.com/amrmagdy16/AI2_Assignment

Domain and planning problem

A team of mobile rescue robots searches a building modelled as a graph of rooms. The robots carry **different sensors and no single robot has all of them**: *thermal* finds a heat signature (detected), *visual* confirms a victim (identified), and *gas* sweeps a room for hazard (gas-checked). A victim is rescued only after this full chain is completed in its room, so the robots must **cooperate** — the right sensor must be routed to the right room in the right causal order. Sensing capability is what constrains the plan, which is the core idea of the assignment.

Modelling choices

Q1 (classical PDDL, BFWS). State is a set of predicates. Ground-truth `victim-in` is given, but knowledge is *produced* by sensing actions that add belief predicates (detected / identified / gas-checked) — the standard PDDL approximation of perception. `rescue` requires `identified` **and** `gas-checked`, so the heterogeneous chain is enforced through preconditions rather than explicit ordering. Two instances are provided: (A) one fully-equipped robot — sensing imposes order but needs no coordination (5-action plan); (B) three single-sensor robots and two victims — all three must converge on each room, making cooperation mandatory (20-action plan).

Q2 (PDDL+, ENHSP). The hazard becomes dynamic. A **process** `gas-buildup` raises gas continuously (`* #t 1`) in active sources; a **process** `gas-ventilation` lets a gas robot pump it back down (`* #t 2`). An **event** `gas-spreads` seeps gas into a clean neighbour at level 8 (it sets `gas-source` / clears `no-gas-source`, deleting its own precondition so it fires once per edge). A **failure event** `victim-lost` makes a victim unrecoverable at level ≥ 10 . Crucially, `rescue` now also tests the **live** numeric condition (`< (gas-level ?l) 5`): a gas-sweep is only a snapshot, so because the gas evolves the planner must reason about *when* a room is safe, not just whether it was checked — this is the sensing–dynamics coupling. Instance A forces the planner to *wait* while ventilation lowers a room below the safe limit ('doing nothing has an effect'); instance B runs an active source toward the lethal threshold while the single gas robot is occupied elsewhere, so the work must be sequenced before the failure event fires.

Plan output (does it behave as expected?)

Q1-A yields `move` → `thermal-scan` → `visual-identify` → `gas-sweep` → `rescue` (5 steps), exactly the sensing chain. **Q1-B** yields a 20-step plan in which `thermalbot`, `visualbot` and `gasbot` each travel to `roomA` and `roomB` and execute their own sensor action before each rescue — confirming cooperation is forced. **Q2-A** performs all sensing at $t=0$, switches on ventilation, then inserts an explicit `-----waiting-----` [1.0] step while `gas-level roomA` falls from 6 to 4 (rate 2/s), and only then rescues at $t=1.0$ — the dynamics engaged exactly as designed. **Q2-B** shows the same wait while it juggles two victims, and both are rescued. Behaviour matches the design. (All plans are committed under `codes/outputs/.`)

Limitations

Sensing is modelled as deterministic, perfect, instantaneous knowledge — real perception is noisy and partial, which would require a POMDP / belief-space formulation. Gas 'spread' is a discrete edge event

rather than true diffusion. Ventilation is fire-and-forget and robot motion costs no time, so timing pressure comes only from the processes. These are deliberate abstractions: the power of the model comes from what it hides — and so does its danger. To make instance B infeasible (forcing genuine victim prioritisation) the lethal threshold can be lowered from 10 to about 6, or the buildup rate raised.